```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_curve,roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
X,y = make_classification(n_samples=1000,n_features=10, random_state=42)
x_train,x_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

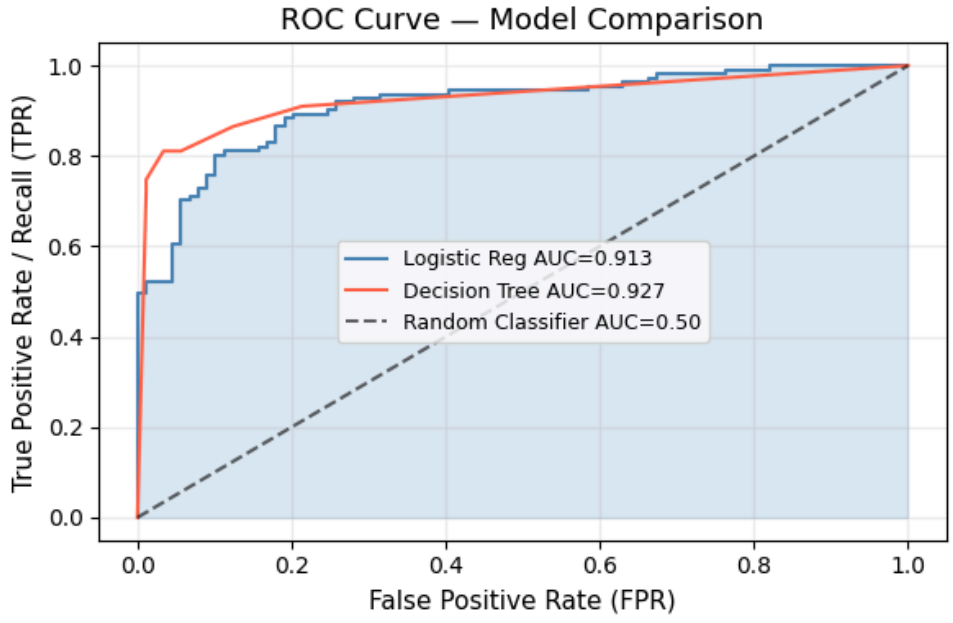
```
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

```
Ir = LogisticRegression().fit(x_train,y_train)
proba_Ir=Ir.predict_proba(x_test)[:,1]
```

```
dt = DecisionTreeClassifier(max_depth=4).fit(x_train,y_train)
proba_dt=dt.predict_proba(x_test)[:,1]
```

```
fpr_lr,tpr_lr,_=roc_curve(y_test,proba_Ir)
fpr_dt,tpr_dt,_=roc_curve(y_test,proba_dt)
auc_lr=roc_auc_score(y_test,proba_Ir)
auc_dt=roc_auc_score(y_test,proba_dt)
```

```
plt.figure(figsize=(6, 4))
plt.plot(fpr_lr, tpr_lr, lw=1.5, color='steelblue', label=f'Logistic Reg AUC={auc_lr:.3f}')
plt.plot(fpr_dt, tpr_dt, lw=1.5, color='tomato', label=f'Decision Tree AUC={auc_dt:.3f}')
plt.plot([0,1],[0,1], 'k--', alpha=0.6, label='Random Classifier AUC=0.50')
plt.fill_between(fpr_lr, tpr_lr, alpha=0.2, color='steelblue')
plt.xlabel('False Positive Rate (FPR)', fontsize=11)
plt.ylabel('True Positive Rate / Recall (TPR)', fontsize=11)
plt.title('ROC Curve — Model Comparison', fontsize=13)
plt.legend(fontsize=9); plt.grid(alpha=0.2)
plt.tight_layout(); plt.show()
```




```
print(f'Logistic Regression AUC: {auc_lr:.4f}')
print(f'Decision Tree AUC: {auc_dt:.4f}')
```

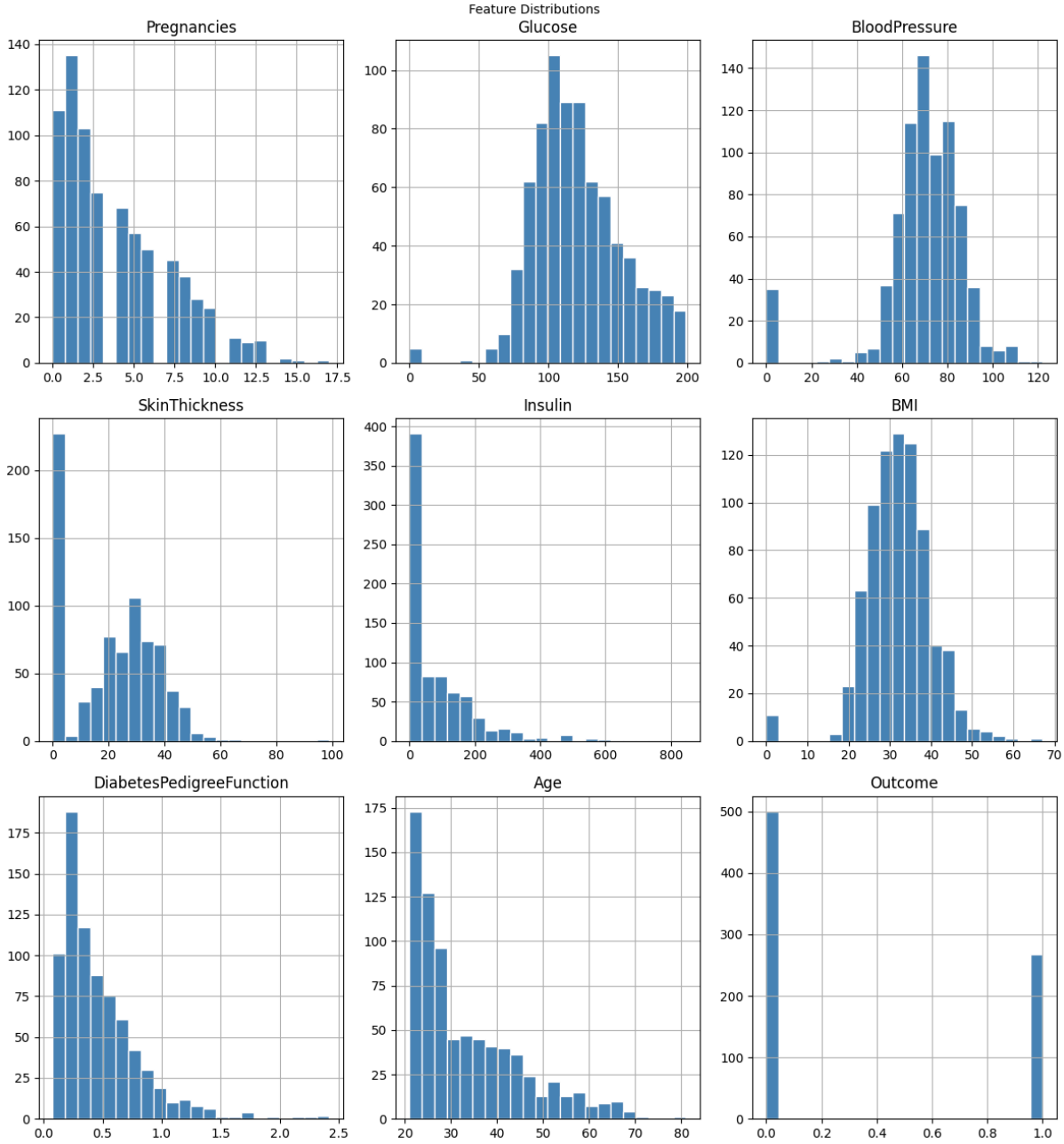
```
print('Higher AUC = better model at separating classes')
```

```
Logistic Regression AUC: 0.9126
Decision Tree AUC: 0.9271
Higher AUC = better model at separating classes
```

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, confusion_matrix,
ConfusionMatrixDisplay, roc_auc_score, roc_curve)
!test -f diabetes.csv || wget https://raw.githubusercontent.com/plotly/datasets/master/diabetes.c
df = pd.read_csv('diabetes.csv')
print(df.shape)
print(df['Outcome'].value_counts())
print(df.describe().round(2))
df.hist(bins=22, figsize=(12,13), color='steelblue', edgecolor='white')
plt.suptitle('Feature Distributions', fontsize=10); plt.tight_layout(); plt.show()
```

```
(768, 9)
Outcome
0    500
1    268
Name: count, dtype: int64
       Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin     BMI  \
count       768.00   768.00         768.00         768.00   768.00  768.00
mean          3.85   120.89          69.11          20.54    79.80   31.99
std           3.37    31.97          19.36          15.95   115.24    7.88
min           0.00     0.00           0.00           0.00     0.00    0.00
25%           1.00    99.00          62.00           0.00     0.00   27.30
50%           3.00   117.00          72.00          23.00    30.50   32.00
75%           6.00   140.25          80.00          32.00   127.25   36.60
max          17.00   199.00         122.00          99.00   846.00   67.10

       DiabetesPedigreeFunction     Age  Outcome
count                    768.00  768.00   768.00
mean                       0.47   33.24     0.35
std                        0.33   11.76     0.48
min                        0.08   21.00     0.00
25%                        0.24   24.00     0.00
50%                        0.37   29.00     0.00
75%                        0.63   41.00     1.00
max                        2.42   81.00     1.00
```



```
fix_cols = ['Glucose','BloodPressure','SkinThickness','Insulin','BMI']
for col in fix_cols:
    n = (df[col] == 0).sum()
    df[col] = df[col].replace(0, df[col].median())
    print(f'{col:25s}: {n} zeros → replaced with median {df[col].median():.1f}')
```

```
Glucose                   : 5 zeros → replaced with median 117.0
BloodPressure             : 35 zeros → replaced with median 72.0
SkinThickness             : 227 zeros → replaced with median 23.0
Insulin                   : 374 zeros → replaced with median 31.2
BMI                       : 11 zeros → replaced with median 32.0
```

```
X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.20, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)
for name, coef in zip(X.columns, model.coef_[0]):
    print(f'{name:30s}: {coef:+.3f}')
```

```
Pregnancies                   : +0.376
Glucose                       : +1.209
BloodPressure                 : -0.054
SkinThickness                 : +0.029
Insulin                       : -0.133
BMI                           : +0.701
DiabetesPedigreeFunction      : +0.245
Age                           : +0.138
```

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
print(classification_report(y_test, y_pred, target_names=['Healthy','Diabetic']))
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm, display_labels=['Healthy','Diabetic']).plot(cmap='Blues')
plt.title('Diabetes Prediction — Confusion Matrix'); plt.show()
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)
plt.figure(figsize=(5.5, 3.5))
plt.plot(fpr, tpr, lw=2.5, color='steelblue', label=f'AUC = {auc:.3f}')
plt.plot([0,1],[0,1],'k--'); plt.xlabel('FPR'); plt.ylabel('TPR')
plt.title('Diabetes Model ROC Curve'); plt.legend(); plt.tight_layout(); plt.show()
```